Практичне заняття №10

Тема : Динамічні структури даних. Конструктори, деструктори і оброблення виняткових ситуацій напиши що потрісбно тут зробить

Виконав: Шалівський Віталій

Група: alk-43

Київ 2026 р

Завдання 1

Мета роботи

Навчитися створювати та використовувати однозв’язні списки, а також реалізовувати базові операції над ними.

Однозв’язний список — це динамічна структура даних, що складається з вузлів. Кожен вузол містить:

- дані (інформаційна частина);
- покажчик на наступний елемент списку.

Список зберігається у пам’яті не послідовно, а через зв’язки між елементами.

Хід роботи

У програмі було:

1. Оголошено структуру вузла списку Node, яка містить поле data та покажчик next.
2. Реалізовано функцію CreateList(), яка:
    - створює список;
    - додає елементи у кінець списку.
3. Реалізовано функцію ShowList(), яка:
    - проходить по списку;
    - виводить всі елементи на екран.

Код програми

```cpp
#include <iostream>
#include <windows.h>

using namespace std;

// ===== СТРУКТУРА ВУЗЛА =====
struct Node {
    int data;
    Node* next;
};

// ===== СТВОРЕННЯ СПИСКУ =====
Node* CreateList() {
    Node* head = NULL;
    Node* tail = NULL;

    int n, value;
    cout << "Введіть кількість елементів: ";
    cin >> n;

    for (int i = 0; i < n; i++) {
        cout << "Введіть елемент: ";
        cin >> value;

        Node* temp = new Node;
        temp->data = value;
        temp->next = NULL;

        if (head == NULL) {
            head = tail = temp;
        }
        else {
            tail->next = temp;
            tail = temp;
        }
    }
```

```cpp
    return head;
}

// ===== ВИВЕДЕННЯ СПИСКУ =====
void ShowList(Node* head) {
    Node* temp = head;

    cout << "Список: ";
    while (temp != NULL) {
        cout << temp->data << " ";
        temp = temp->next;
    }
    cout << endl;
}

// ===== MAIN =====
int main() {
    SetConsoleCP(1251);
    SetConsoleOutputCP(1251);

    Node* head = CreateList();
    ShowList(head);

    return 0;
}
```

Результат виконання програми

Користувач вводить кількість елементів та їх значення.

Програма створює список і виводить його на екран у вигляді послідовності чисел.

```
Введіть кількість елементів: 3
Введіть елемент: 10
Введіть елемент: 20
Введіть елемент: 30
Список: 10 20 30

D:\Desktop\студент\ООП\На 20 04 26\ооп завдання 1\x64\Debug\ооп завдання 1.exe (process 12896) exited with code 0 (0x0).
Press any key to close this window . . .
```

Висновок

У ході виконання роботи було освоєно створення однозв’язного списку, а також реалізовано функції додавання та виведення елементів. Це дозволяє ефективно працювати з динамічними структурами даних.

Завдання 2

Мета роботи

Навчитися працювати з однозв’язними списками та реалізовувати основні операції над ними за допомогою меню.

Теоретичні відомості

Однозв’язний список — це динамічна структура даних, що складається з вузлів. Кожен вузол містить дані та покажчик на наступний елемент. Така структура дає можливість змінювати кількість елементів під час виконання програми.

Хід роботи

У програмі було створено однозв’язний список і реалізовано меню для виконання різних дій над ним. Користувач може створити список, переглянути його, видаляти елементи з початку, з кінця або за ключем, а також додавати нові елементи на початок, у кінець або після заданого ключа. Після завершення роботи список повністю очищується.

Код програми

ооп завдання 2.cpp | What's New?

ооп завдання 2 | (Global Scope) | main()

```cpp
#include <iostream>
#include <windows.h>

using namespace std;

struct Node {
    int data;
    Node* next;
};

// створення списку
void CreateList(Node*& head) {
    int n, value;
    cout << "Введіть кількість елементів: ";
    cin >> n;

    head = NULL;
    Node* tail = NULL;

    for (int i = 0; i < n; i++) {
        cout << "Введіть елемент " << i + 1 << ": ";
        cin >> value;

        Node* temp = new Node;
        temp->data = value;
        temp->next = NULL;

        if (head == NULL) {
            head = temp;
            tail = temp;
        }
        else {
            tail->next = temp;
            tail = temp;
        }
    }
}
```

```cpp
// перегляд списку
void ShowList(Node* head) {
    if (head == NULL) {
        cout << "Список порожній." << endl;
        return;
    }

    cout << "Список: ";
    Node* temp = head;
    while (temp != NULL) {
        cout << temp->data << " ";
        temp = temp->next;
    }
    cout << endl;
}

// видалення з початку
void DeleteFromBegin(Node*& head) {
    if (head == NULL) {
        cout << "Список порожній." << endl;
        return;
    }

    Node* temp = head;
    head = head->next;
    delete temp;

    cout << "Перший елемент видалено." << endl;
}
```

```cpp
// видалення з кінця
void DeleteFromEnd(Node*& head) {
    if (head == NULL) {
        cout << "Список порожній." << endl;
        return;
    }

    if (head->next == NULL) {
        delete head;
        head = NULL;
        cout << "Останній елемент видалено." << endl;
        return;
    }

    Node* temp = head;
    while (temp->next->next != NULL) {
        temp = temp->next;
    }

    delete temp->next;
    temp->next = NULL;

    cout << "Останній елемент видалено." << endl;
}

// видалення за ключем
void DeleteByKey(Node*& head, int key) {
    if (head == NULL) {
        cout << "Список порожній." << endl;
        return;
    }
```

```cpp
    while (head != NULL && head->data == key) {
        Node* temp = head;
        head = head->next;
        delete temp;
    }

    Node* current = head;
    while (current != NULL && current->next != NULL) {
        if (current->next->data == key) {
            Node* temp = current->next;
            current->next = temp->next;
            delete temp;
        }
        else {
            current = current->next;
        }
    }

    cout << "Видалення за ключем завершено." << endl;
}

// додавання на початок
void AddToBegin(Node*& head, int value) {
    Node* temp = new Node;
    temp->data = value;
    temp->next = head;
    head = temp;

    cout << "Елемент додано на початок." << endl;
}
```

```cpp
// додавання в кінець
void AddToEnd(Node*& head, int value) {
    Node* temp = new Node;
    temp->data = value;
    temp->next = NULL;

    if (head == NULL) {
        head = temp;
    }
    else {
        Node* current = head;
        while (current->next != NULL) {
            current = current->next;
        }
        current->next = temp;
    }

    cout << "Елемент додано в кінець." << endl;
}

// додавання після ключа
void AddAfterKey(Node*& head, int key, int value) {
    if (head == NULL) {
        cout << "Список порожній." << endl;
        return;
    }

    Node* current = head;
    while (current != NULL && current->data != key) {
        current = current->next;
    }
```

```cpp
    if (current == NULL) {
        cout << "Ключ не знайдено." << endl;
        return;
    }

    Node* temp = new Node;
    temp->data = value;
    temp->next = current->next;
    current->next = temp;

    cout << "Елемент додано після ключа." << endl;
}

// очищення списку
void ClearList(Node*& head) {
    while (head != NULL) {
        Node* temp = head;
        head = head->next;
        delete temp;
    }
    cout << "Список очищено." << endl;
}

int main() {
    SetConsoleCP(1251);
    SetConsoleOutputCP(1251);

    Node* head = NULL;
    int choice, value, key;
```

```cpp
do {
    cout << "\n===== МЕНЮ =====" << endl;
    cout << "1. Створити список" << endl;
    cout << "2. Переглянути список" << endl;
    cout << "3. Видалити елемент з початку" << endl;
    cout << "4. Видалити елемент за ключем" << endl;
    cout << "5. Видалити елемент з кінця" << endl;
    cout << "6. Додати елемент на початок" << endl;
    cout << "7. Додати елемент в кінець" << endl;
    cout << "8. Додати елемент після ключа" << endl;
    cout << "9. Вихід та очищення списку" << endl;
    cout << "Ваш вибір: ";
    cin >> choice;

    switch (choice) {
    case 1:
        ClearList(head);
        CreateList(head);
        break;
    case 2:
        ShowList(head);
        break;
    case 3:
        DeleteFromBegin(head);
        break;
    case 4:
        cout << "Введіть ключ для видалення: ";
        cin >> key;
        DeleteByKey(head, key);
        break;
    case 5:
        DeleteFromEnd(head);
        break;
    case 6:
```

```cpp
        cout << "Введіть значення: ";
        cin >> value;
        AddToBegin(head, value);
        break;
    case 7:
        cout << "Введіть значення: ";
        cin >> value;
        AddToEnd(head, value);
        break;
    case 8:
        cout << "Введіть ключ, після якого додати: ";
        cin >> key;
        cout << "Введіть значення нового елемента: ";
        cin >> value;
        AddAfterKey(head, key, value);
        break;
    case 9:
        ClearList(head);
        cout << "Програма завершена." << endl;
        break;
    default:
        cout << "Неправильний вибір." << endl;
    }

} while (choice != 9);

return 0;
}
```

Результат виконання програми

```
D:\Desktop\студент\ООП\На

===== МЕНЮ =====
1. Створити список
2. Переглянути список
3. Видалити елемент з початку
4. Видалити елемент за ключем
5. Видалити елемент з кінця
6. Додати елемент на початок
7. Додати елемент в кінець
8. Додати елемент після ключа
9. Вихід та очищення списку
Ваш вибір: 1
Список очищено.
Введіть кількість елементів: 10
Введіть елемент 1: 1
Введіть елемент 2: 2
Введіть елемент 3: 3
Введіть елемент 4: 4
Введіть елемент 5: 5
Введіть елемент 6: 6
Введіть елемент 7: 7
Введіть елемент 8: 8
Введіть елемент 9: 9
Введіть елемент 10: 10
```

```
===== МЕНЮ =====
1. Створити список
2. Переглянути список
3. Видалити елемент з початку
4. Видалити елемент за ключем
5. Видалити елемент з кінця
6. Додати елемент на початок
7. Додати елемент в кінець
8. Додати елемент після ключа
9. Вихід та очищення списку
Ваш вибір: 2
Список: 1 2 3 4 5 6 7 8 9 10

===== МЕНЮ =====
1. Створити список
2. Переглянути список
3. Видалити елемент з початку
4. Видалити елемент за ключем
5. Видалити елемент з кінця
6. Додати елемент на початок
7. Додати елемент в кінець
8. Додати елемент після ключа
9. Вихід та очищення списку
Ваш вибір: 3
Перший елемент видалено.
```

```
===== МЕНЮ =====
1. Створити список
2. Переглянути список
3. Видалити елемент з початку
4. Видалити елемент за ключем
5. Видалити елемент з кінця
6. Додати елемент на початок
7. Додати елемент в кінець
8. Додати елемент після ключа
9. Вихід та очищення списку
Ваш вибір: 2
Список: 2 3 4 5 6 7 8 9 10

===== МЕНЮ =====
1. Створити список
2. Переглянути список
3. Видалити елемент з початку
4. Видалити елемент за ключем
5. Видалити елемент з кінця
6. Додати елемент на початок
7. Додати елемент в кінець
8. Додати елемент після ключа
9. Вихід та очищення списку
Ваш вибір: 7
Введіть значення: 100
Елемент додано в кінець.
```

```
===== МЕНЮ =====
1. Створити список
2. Переглянути список
3. Видалити елемент з початку
4. Видалити елемент за ключем
5. Видалити елемент з кінця
6. Додати елемент на початок
7. Додати елемент в кінець
8. Додати елемент після ключа
9. Вихід та очищення списку
Ваш вибір: 2
Список: 2 3 4 5 6 7 8 9 10 100

===== МЕНЮ =====
1. Створити список
2. Переглянути список
3. Видалити елемент з початку
4. Видалити елемент за ключем
5. Видалити елемент з кінця
6. Додати елемент на початок
7. Додати елемент в кінець
8. Додати елемент після ключа
9. Вихід та очищення списку
Ваш вибір:
```

У програмі було реалізовано однозв’язний список з використанням динамічної пам’яті.

Створено меню для роботи зі списком, яке дозволяє виконувати різні операції: створення, перегляд, додавання та видалення елементів.

Після вибору пункту «Створити список» користувач вводить кількість елементів та їх значення.
Програма формує список, додаючи елементи у кінець.

При виборі пункту «Переглянути список» відбувається виведення всіх елементів списку на екран.

Завдання 3

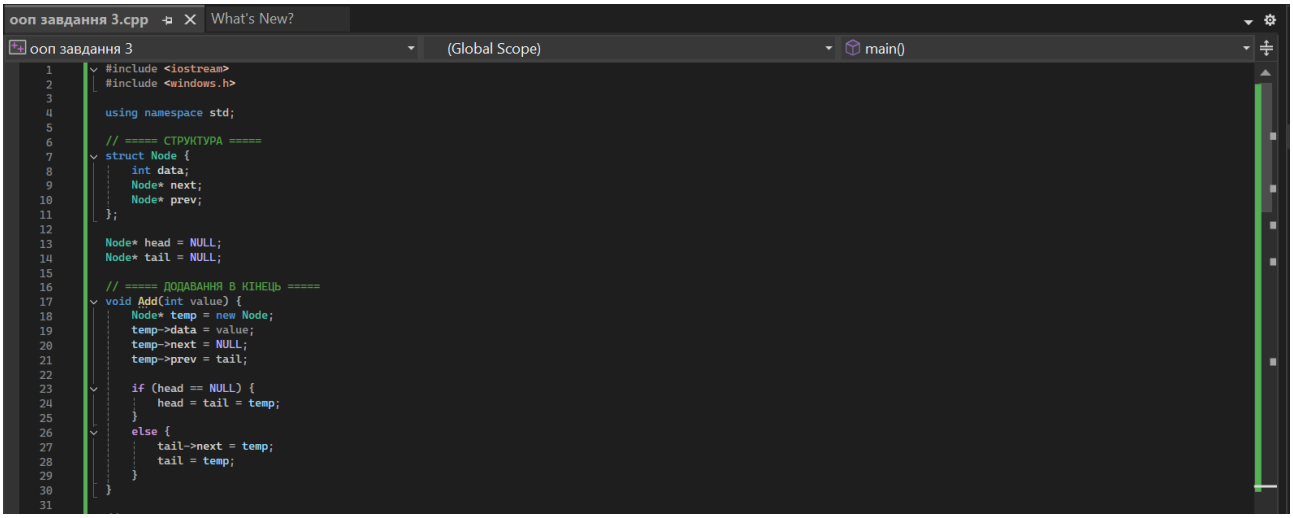
Хід роботи

У програмі було реалізовано двозв’язний список, кожен елемент якого містить покажчики на наступний та попередній вузли.

Було створено функції для додавання елементів у список, перегляду списку з початку та з кінця, видалення елемента за значенням та очищення списку.

Після введення елементів список було виведено у прямому та зворотному порядку. Далі виконано видалення заданого елемента та повторний перегляд списку.

Код програми




```
#include <iostream>
#include <windows.h>

using namespace std;

// ===== СТРУКТУРА =====
struct Node {
    int data;
    Node* next;
    Node* prev;
};

Node* head = NULL;
Node* tail = NULL;

// ===== ДОДАВАННЯ В КІНЕЦЬ =====
void Add(int value) {
    Node* temp = new Node;
    temp->data = value;
    temp->next = NULL;
    temp->prev = tail;

    if (head == NULL) {
        head = tail = temp;
    }
    else {
        tail->next = temp;
        tail = temp;
    }
}
```

```cpp
// ===== ВИВІД З ПОЧАТКУ =====
void ShowForward() {
    Node* temp = head;
    cout << "Список з початку: ";
    while (temp != NULL) {
        cout << temp->data << " ";
        temp = temp->next;
    }
    cout << endl;
}

// ===== ВИВІД З КІНЦЯ =====
void ShowBackward() {
    Node* temp = tail;
    cout << "Список з кінця: ";
    while (temp != NULL) {
        cout << temp->data << " ";
        temp = temp->prev;
    }
    cout << endl;
}

// ===== ВИДАЛЕННЯ ЕЛЕМЕНТА =====
void Delete(int value) {
    Node* temp = head;

    while (temp != NULL) {
        if (temp->data == value) {

            if (temp == head) {
                head = temp->next;
                if (head != NULL) head->prev = NULL;
            }
            else if (temp == tail) {
                tail = temp->prev;
                if (tail != NULL) tail->next = NULL;
```

```cpp
            }
            else {
                temp->prev->next = temp->next;
                temp->next->prev = temp->prev;
            }

            delete temp;
            cout << "Елемент видалено!\n";
            return;
        }
        temp = temp->next;
    }

    cout << "Елемент не знайдено!\n";
}

// ===== ОЧИЩЕННЯ =====
void Clear() {
    Node* temp = head;
    while (temp != NULL) {
        Node* next = temp->next;
        delete temp;
        temp = next;
    }
    head = tail = NULL;
}

// ===== MAIN =====
int main() {
    SetConsoleCP(1251);
    SetConsoleOutputCP(1251);

    int n, x;

    cout << "Скільки елементів: ";
    cin >> n;
```

```cpp

    for (int i = 0; i < n; i++) {
        cout << "Введіть елемент: ";
        cin >> x;
        Add(x);
    }

    ShowForward();
    ShowBackward();

    cout << "Який елемент видалити: ";
    cin >> x;
    Delete(x);

    ShowForward();

    Clear();

    return 0;
}
```

Результат виконання програми

```
Microsoft Visual Studio Debug

Скільки елементів: 5
Введіть елемент: 1 2 3 4 5
Введіть елемент: Введіть елемент: Введіть елемент: Введіть елемент: Список з початку: 1 2 3 4 5
Список з кінця: 5 4 3 2 1
Який елемент видалити: 3
Елемент видалено!
Список з початку: 1 2 4 5

D:\Desktop\студент\ООП\На 20 04 26\ооп завдання 3\x64\Debug\ооп завдання 3.exe (process 7560) exited with code 0 (0x0).
Press any key to close this window . . .
```

На даному етапі було створено двозв’язний список та виконано його перегляд з початку та з кінця. Це підтверджує правильність роботи двонаправлених зв’язків між елементами списку.

Висновок

я, перегляду, видалення та очищення списку. Програма коректно працює з динамічною пам’яттю та демонструє переваги двозв’язних структур даних.

Контрольні питання

№	Питання	Відповідь
1	Які основні положення системи уведення-виведення C++?	У C++ використовується потокова модель введення-виведення. Дані передаються через потоки (istream, ostream). Основні об’єкти: cin (введення), cout (виведення), cerr (помилки), clog (логування). Використовуються оператори << (виведення) і >> (введення).
2	Що таке потік?	Потік — це абстракція для передачі даних між програмою і зовнішнім середовищем (екран, файл, клавіатура).
3	Які потоки відкриваються при запуску програми?	cin — введення (клавіатура); cout — виведення (екран); cerr — потік помилок (без буферизації); clog — потік логування (з буферизацією).
4	Які прапори формату ви знаєте?	ios::left, ios::right — вирівнювання; ios::dec, ios::hex — система числення; ios::fixed — фіксований формат чисел.
5	Які функції форматування ви знаєте?	setw() — ширина поля; setprecision() — точність; setfill() — символ заповнення.
6	Які особливості файлового	Для роботи з файлами використовуються потоки ifstream (читання), ofstream (запис), fstream

	введення-виведення в C++?	(читання/запис). Потрібно відкривати і закривати файли.
7	Які функції виконують оператори << і >>?	<< — виводить дані в потік (cout); >> — зчитує дані з потоку (cin).
8	Як створюються функції вставки та вилучення?	Перевантажуються оператори << і >> для класів через friend-функції.
9	Яка загальна форма функції вставки?	ostream& operator<<(ostream& os, const Class& obj) { ... return os; }
10	Яка загальна форма функції вилучення?	istream& operator>>(istream& is, Class& obj) { ... return is; }
11	Чи можуть ці функції належати класу?	Ні, зазвичай це зовнішні (friend) функції.
12	Які параметри використовуються?	Для << — ostream& і const об’єкт; для >> — istream& і об’єкт без const.